

BOAZ 분석 26 기 조수빈 week9 < RAG & LoRA >

[RAG: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks]

사전 학습된 언어 모델들은 매개변수 내에 엄청난 양의 사실인 지식을 저장할 수 있지만, 지식에 접근하고 정밀하게 조작하는 능력에는 한계가 있었다. 모델이 내린 결정의 출처를 정확히 알기 어렵고, 지식이 변할 때마다 모델을 업데이트하는 것도 어려웠다. 기존의 검색 후 추출 방식은 정보 출처를 제공할 수는 있으나, 주어진 텍스트 내에서 정답 구간만 추출하기 때문에 자유로운 텍스트 생성이 불가능하다는 단점이 있었다.

검색기와 생성기를 End-to-End 로 학습시키는 검색 증강 생성(RAG) 모델을 설계했다.

검색기 (Retriever) : 비모수적 메모리인 위키피디아 문서 인덱스에 접근한다. DPR(Dense Passage Retriever)이라는 이중 인코더(Bi-encoder) 아키텍처를 사용하여 주어진 질문과 문서 간의 유사도를 Maximum Inner Product Search (MIPS)로 계산하여 상위 K 개의 문서를 추출하게 된다.

생성기 (Generator) : 사전 학습된 seq2seq 트랜스포머 모델인 BART-large 를 매개변수 메모리로 사용한다. 입력 쿼리와 검색기가 찾은 문서를 결합하여 최종 텍스트를 생성하게 된다.

잠재 변수 주변화 (Marginalization) : 검색된 문서를 잠재 변수로 취급하여 예측 확률을 계산하는 두 가지 방식을 제안한다.

RAG-Sequence: 생성할 전체 텍스트 시퀀스에 대해 동일한 단일 문서를 참고하여 확률을 계산한다.

RAG-Token: 텍스트를 생성할 때 각 토큰마다 다른 문서를 유연하게 참고할 수 있도록 토큰 단위로 확률을 주변화(Marginalize)한다.

훈련 시에는 모델이 '어떤 문서'를 검색해야 하는지에 대한 직접적인 레이블이나 지도 신호를 주지 않는다. 입력과 정답 쌍이 주어지면, 정답을 생성할 전체적인 음의 Negative marginal log-likelihood 를 최소화하도록 검색기(Query Encoder)와 생성기(BART)를 동시에 최적화한다. 문서 인덱스를 인코딩하는 Document Encoder 는 훈련 과정에서 고정하여 연산 비용도 감소했다.

RAG 연구에서 사용된 데이터와 성과는 다음과 같다.

데이터 : 2018 년 12 월 위키피디아 덤프를 100 단어 단위로 쪼갠 2,100 만 개의 문서를 비모수적 메모리로 사용했다. 평가를 위해 Natural Questions, TriviaQA, WebQuestions 등 다양한 질의응답 및 사실 검증데이터셋을 활용했다.

성과 : 4 개의 오픈 도메인 QA 벤치마크 중 3 개에서 SOTA 을 갱신하게 된다. 단순 추출형 기반 모델들보다 기능이 좋았고, BART 에 비해 환각이 적고 사실적이며 다양한 문장을 생성했다. 재학습 없이 위키피디아 인덱스만 2016 년 버전에서 2018 년 버전으로 교체하여 모델의 세계 지식을 즉각적으로 업데이트 했다.

RAG-Sequence 모델의 경우, 생성된 가설 텍스트가 여러 검색 문서의 범에 공통으로 등장하지 않을 경우 추가적인 순방향 패스 연산이 필요하여 디코딩 효율성이 떨어지는 한계가 있다. 외부 지식 베이스가 완벽한 사실만을 담고 있지 않거나 편향을 가질 경우 모델이 이를 그대로 반영할 위험이 있다.

1. Background

1. 기존 LLM 은 다음과 같은 문제가 있었다.
2. 메모리 확장 어려움, 수정 불가
3. 모든 지식을 파라미터에 저장하기 어려움
4. 할루시네이션을 생성
5. 해결 방안 : 외부 문서 데이터베이스에서 관련 정보를 검색한 뒤 LLM 과 결합하여 답변 생성하는 RAG

2. Related Work

1. Single-task Retrieval
2. General-purpose NLP ARchitectures
3. Learned Retrieval
4. Memory-based Architectures
5. Retrieve-and-Edit

⇒ RAG : Retrieval + Generative LLM 을 결합하여 여러 knowledge-intensive NLP task 를 하나의 architecture 로 해결

3. Introduction

1. RAG 는 검색기와 생성기
2. 검색기 input : 쿼리 x 가 주어졌을 때 텍스트 구 절에 대한 (top-k 로 자른) 분포를 반환하 는 파라미터 n 를 가짐
3. 생성기 이전 1-1 개의 토큰 $y_1 : 1-1$, 원본 입력 x 및 검색 된 구절 z 의 컨텍스트를 기반으로 현재 토큰을 생성하는 파라미터 0 를 가짐

4. RAG 의 Hybrid Memory

1. 모수적 메모리 : 일반적인 사전 학습된 seq2seq 모델처럼 이미 학습된 파라미터 안에 지식을 저장하는 방식 - BART
2. 비모수적 메모리 : 검색 간으한 외부 데이터베이스를 벡터 데이터베이스로 구성하여 필요한 지식을 검색하고 활용 - Wikipedia

5. Generator : BART

1. 모델: 4 억 파라미터의 사전 학습된 seq2seq 트랜스포머 (BART-large)
2. 입력 구성: 쿼리 x 와 검색된 문서 Z 를 단순 연결하여 입력
3. 사전 학습 방식: 다양한 노이즈 함수를 활용한 디노이징으로 훈련
4. RAG 에 적합한 이유: 인코더-디코더 구조로 검색된 문서를 조건으로 자연스러운 텍스트 생성 가능
5. 성능: 동일 크기 TS 모델 대비 다양한 생성 태스크에서 우수한 성능

6. Training

1. 학습 대상: 쿼리 인코더 BERT + BART 생성기 (end-to-end fine-tuning)
2. 문서 인코더(BERTd) 및 인덱스는 고정학습 중 업데이트 시 문서 인덱스를 매번 재구 축해야 함
3. 비용 대비 성능 향상이 미미하여 고정으로 결정

7. Decoding

1. RAG-Sequence Model

1. 전체 시퀀스 단위 생성
2. 검색된 문서 하나를 시퀀스 전체에 동일하게 사용
3. 상위 K 개 문서 검색
4. 각 문서로 전체 출력 시퀀스 확률 계산
5. 확률 marginalize - 최종 답변 생성

2. RAG-Token Model

1. 토큰 단위의 유연한 생성
2. 토큰 생성마다 다른 문서 참조 가능
3. 상위 K 개 문서 검색
4. 각 토큰마다 문서별 분포 생성
5. 분포 marginalize - 다음 토큰 생성 - 반복

8. Experiments & Results

9. Experiment Setting

1. Document - Wikipedia dump → Embedding FAISS (facebook vector library) → Vector Search MIPS (Maximum Inner product search)
2. 이때 FAISS 는 조건값

10. Experiments & Results

1. Open domain QA : 광범위한 주제에 걸친 질문에 대한 답
2. Abstractive QA : 문서 추출이 아닌 답변을 생성
3. Jeopardy : 주어진 정답 기반으로 문장 생성 여부
4. Fact Verification : 어떤 주장이 참인지 거짓인지 판단
 1. evidence 안 알려줌
 2. RAG 는 검색과 결과를 도출
 3. SOTA 에 비해 아직 부족하지만 스스로 판단
 4. Open Domain : k 가 너무 많으면 성능 down
 5. Recall : k 가 커질수록 recall 향상
 6. 생성 문장의 평가 점수

11. Conclusion

1. 의미 : RAG 텍스트 생성은 모델 파라미터에 저장된 지식, 외부 문서 검색
2. 향후 연구 1 : parametric, non-parametric memory 가 어떻게 상호작용 하는지, 두가지를 어떤 방식을 통해 효율적으로 연결할 수 있는지
3. 향후 연구 2 : retriever 와 generator 를 BART 의 denoising functiond 이나 다른 목적 함수로 처음부터 pretraining 할 수 있는지

12. Follow-up work

1. 다단계 RAG : 쿼리를 반복 재구성해 이전 검색 결과를 바탕으로 점진적 답변 생성
2. 대화형 RAG : 과거 대화 맥락 & 새로 검색된 정보를 결합해 일관된 멀티턴 응답 생성
3. 하이브리드 및 그래프 기반 RAG : 어휘 검색 & 벡터 검색 결합, 지식 그래프로 데이터 간 관계 파악
4. 에이전트 RAG : 가설 수립 → 증거 검색 → 결과 평가를 반복하여 자율적으로 답변 도출

[LoRA: Low-Rank Adaptation of Large Language Models]

사전 학습된 모델이 커지면서 모든 매개변수를 업데이트하는 전체 미세 조정(Full Fine-Tuning)은 하드웨어 자원 측면에서 불가능에 가까워졌다. 1750 억 개의 파라미터를 가진 GPT-3 를 작업마다 미세 조정하여 각각 독립적으로 배포하는 것은 비용적으로 금지되어 있다. 이를 해결하기 위한 기존에 사용하던 Adapter layers 추가 또는 Prefix tuning 방식은 모델의 깊이를 늘려 추론 시 지연 시간을 증가시키거나, 사용 가능한 시퀀스 길이를 감소시켜 모델의 품질을 저하시킨다는 단점이 있다.

LoRA 는 Adaptation 중에 발생하는 가중치의 변화량은 매우 낮은 본질적 랭크를 가진다는 가설을 가진다.

Reparametrization : 사전 학습된 거대 가중치 행렬 W_0 는 고정된 채, 업데이트할 변화량 ΔW 를 두 개의 저랭크 행렬 B 와 A 의 곱($\Delta W=BA$)으로 근사하여 훈련한다. 이때 A 는 랜덤 가우시안으로, B 는 0 으로 초기화하여 학습 시작 시점에는 $\Delta W=0$ 이 되도록 만들게 된다.

No Inference Latency : 기존의 어댑터 층과 달리, LoRA 는 실제 서비스 시 학습된 행렬 BA 를 원래의 가중치 W_0 에 직접적으로 더하는 ($W=W_0+BA$) 방식을 사용한다. 따라서 구조가 추가되지 않으며, 완전히 미세 조정된 모델과 비교하여 아주 잠깐의 추가적인 추론 지연도 발생하지 않게 된다.

최적의 가중치와 랭크 : 트랜스포머 아키텍처 내에서 셀프 어텐션 모듈의 가중치(W_q, W_k, W_v, W_o) 중 쿼리(W_q)와 값(W_v) 행렬에만 LoRA 를 적용해도 충분하다는 결과가 나왔다. ΔW 의 랭크 r 이 단 1 이나 2 처럼 극단적으로 작아도 좋은 성능을 유지했다.

RoBERTa, DeBERTa, GPT-2, GPT-3 등 다양한 크기와 종류의 사전 학습 모델에 LoRA 를 적용했습니다. 즉, 자연어 이해(NLU) 성능 평가를 위해 GLUE 벤치마크를 사용했고, 자연어 생성(NLG) 및 Data-to-text 평가를 위해 WikiSQL, SAMSum, E2E NLG Challenge 등의 대규모 데이터셋을 활용했다.

LoRA 는 기존의 전체 미세 조정(Full fine-tuning) 이상의 결과를 가져왔다. 특히 GPT-3 175B 모델에 적용했을 때, 훈련해야 할 매개변수의 수를 전체의 10,000 분의 1 수준으로 줄였다. 훈련 시 GPU VRAM 사용량을 1.2TB 에서 350GB 로 약 3 배 감소시켰으며, 기울기 계산이 생략되므로 훈련 속도도 25% 이상 향상되었다. 배포 시에도 가중치 A 와 B 만 스왑하면 되기때문에 350GB 체크포인트를 작업 개수별로 저장할 필요 없이 약 35MB 크기의 모듈만 저장하면 되기에 스토리지 전환 오버헤드가 감소했다.

$W=W_0+BA$ 구조로 가중치를 병합해버린 경우, 서로 다른 A 와 B 행렬을 요구하는 여러 개의 요청을 하나의 배치로 묶어 동시에 추론하는 것이 어렵다. 또한, 트랜스포머의 여러개의 가중치 행렬 중 어느 부분에 LoRA 를 적용해야 최적의 성능이 나오는지에 대해서는 수학적 증명보다 경험적 휴리스틱에 주로 의존했다는 한계가 있다.

1. Introduction

1. problem : pretrained model 은 모델의 모든 파라미터를 업데이트 해야함 → 저장비용 & 계산 비용 ⇒ downstream task 에 맞게 fine tuning 하고자함
2. 기존에는 Adapter 추가, Prefix tuning 진행

2. Problem statement

1. 기존 방식은 Full fine tuning
2. task 마다 새로운 파라미터가 필요함

3. Aren't you existing solutions good enough?

1. Adaptive layers 를 추가해서 할때에는 GPU 병렬처리 불가능

4. LoRA : Low Rank Weight Update

1. 기본 fine tuning 과 다르게 LoRA 는 weight update 를 Low-rank decomposition 으로 표현
2. LoRA 의 적용 위치는 Transformer Self-Attention, Wq(Query), Wk(Key), Wv(Value), Wo(Output)일때, Wq 와 Wv 에만 적용

5. Empirical experiments

1. Empirical experiments 에서 미리 나온 것과 유사하게 진행.
2. Pretrained 모델 → 특정 task 에 맞게 사용 (일부 Layer 만 학습)
3. Task1 : NLU - 모델이 문장의 의미를 이해하는가?
4. Task2 : NLG - 모델이 새로운 텍스트를 잘 생성하는가?
5. 평가 기준 : GLUE benchmark - 감정분석/ 문장 유사도/ 자연어 추론 평가 기준
6. 실험 사진 : ROBERTA, DEBERTA 는 NLU, GPT-2, GPT-3 는 NLG 평가, 각각 다른 곳에서 한 실험을 하나로 표현

6. Related works

1. Transformer Model : 모델이 커질수록 성능이 좋아짐
2. Adaptation : 기존 layer 사이에 adapter layer 삽입
3. Prompt Engineering Fine-Tuning : 모델의 입력 또는 일부분 학습으로 Task 성능 개선
4. Low-Rank Structures : 많은 ML 문제는 low-rank structure 를 가짐

7. Understanding the low-rank updates

1. Transformer 의 weight matrix 업데이트
2. 실제로 low-rank 구조를 가지는가?
3. 색이 연할수록 유사, 색이 짙을수록 다름
4. 기존 weight 와의 관계 : 가중치 증폭에 대한 결과

8. Conclusion & future work

1. Conclusion

1. Low-rank 기반 parameter-efficient adaptation
2. 추가 latency 없이 적용
3. shared parameter 구조로 빠른 task switching

2. Future Work

1. 다른 PEFT 방법과 결합
2. adaptation mechanism 이해
3. LoRA 적용 layer 선택 문제
4. model weight 의 low-rank 구조 분석